

## A DERIVATION OF THE ORDER SELECTIVITY

The output cardinality  $out(u_S^t)$  of an update node (1) estimates the size of the candidate set for a subquery  $Q|_S$  after batch  $t$  based on the independently filtered rows per variable, the selectivities of the internalized dependent conditions, and the order selectivity  $\Lambda_t(S)$ . The order selectivity reflects the fraction of the remaining row combinations satisfying the pattern's sequential order. Below, we first derive this fraction for the complete candidate set and then for the per-batch increment computed by the update join.

**The Complete Candidate Set.** Let  $S = (V_{a_1}, \dots, V_{a_m})$  with  $m = |S|$  be ordered by pattern position. A candidate combines one row per variable, drawn from the  $|T_t| \varphi_V$  rows satisfying the respective independent condition. We assume the relative ordering of rows bound to different variables to be uniformly distributed, i.e., without further information, each of the  $m!$  orderings of the  $m$  drawn rows is equally likely. Among the  $m!$  orderings of  $m$  distinct rows, exactly one corresponds to the pattern's sequential order. Therefore, the order selectivity of the candidate set is given by:

$$\Lambda_t(S) = \frac{1}{|S|!}. \quad (4)$$

Note that this value does not depend on  $t$ .

**The Per-Batch Increment.** Due to Lemma 1, the update join at batch  $t$  computes only the new candidates instead of the complete candidate set: the last variable of  $S$  draws its rows from the batch  $\mathcal{B}_t$ , while every other variable draws its rows from  $T_t$ , which consists of the history  $T_{t-1}$  and the batch  $\mathcal{B}_t$ . Under append-only semantics, every history row precedes every batch row. A new candidate can therefore satisfy the sequential order only if its history rows occupy a prefix of the variables of  $S$  and its batch rows the remaining suffix; combinations without this structure never satisfy the order. For a prefix of  $p$  history rows, the uniformity assumption applies within each group: the history rows satisfy their order in one of  $p!$  cases, the batch rows in one of  $(m-p)!$  cases, and the order between the two groups holds by arrival. The order selectivity of the increment, denoted by  $\Lambda_t^\Delta(S)$ , therefore sums over all prefix lengths  $p$  the probability that a combination with  $p$  history rows satisfies the order, weighted by the probability of drawing such a combination:

$$\Lambda_t^\Delta(S) = \sum_{p=0}^{m-1} \frac{1}{p! (m-p)!} \cdot \frac{|T_{t-1}|^p |\mathcal{B}_t|^{m-1-p}}{|T_t|^{m-1}}. \quad (5)$$

Here, the first factor reflects the order probability within the two groups. The second factor reflects the probability of drawing a combination whose first  $p$  variables hold history rows: each of the  $m-1$  non-last variables draws a history row with probability  $|T_{t-1}| / |T_t|$  and a batch row with probability  $|\mathcal{B}_t| / |T_t|$ , while the last variable always draws a batch row. Multiplying these ratios yields one factor  $|T_{t-1}|$  per history variable, one factor  $|\mathcal{B}_t|$  per batch-drawing non-last variable, and one factor  $|T_t|$  per non-last variable, resulting in the exponents  $p$ ,  $m-1-p$ , and  $m-1$ , respectively. Hence,  $p = m$  is excluded, and the independent selectivities cancel within these ratios.

For  $t = 1$ , the history  $T_0$  is empty, only the term for  $p = 0$  remains, and (5) reduces to  $1/m!$ , i.e., to (4): on the first batch, the increment is the complete candidate set. For a two-variable view, the sum simplifies to  $\Lambda_t^\Delta(S) = 1 - |\mathcal{B}_t| / (2|T_t|)$ . For constant batch sizes, this value approaches 1 as the table grows, since the batch shrinks

relative to the history and a pair combining a history row with a batch row always satisfies the order. In the running example, the update of  $(A, O)$  joins new higher-fare trips bound to  $O$  with reference trips bound to  $A$ , which mostly precede them. Moreover, the two forms are consistent, which we illustrate for  $m = 2$ . Every ordered pair is produced exactly once, namely at the batch in which its later row arrives (Lemma 1), so the increments must add up to the complete candidate set. At batch  $t$ , the update join processes the  $|T_t| |\mathcal{B}_t|$  pairs of a row from  $T_t$  and a row from  $\mathcal{B}_t$ ; applying (5) and substituting  $|\mathcal{B}_t| = |T_t| - |T_{t-1}|$  yields  $(|T_t|^2 - |T_{t-1}|^2)/2$  ordered pairs. Summing over the batches  $1, \dots, t$  therefore cancels all intermediate terms and yields  $|T_t|^2/2$ , i.e., the  $|T_t|^2$  pairs over the full table reduced by the order selectivity  $1/2!$  of (4).

## B ESTIMATION OF THE COMPOSITION FACTORS

The output cardinality  $out(y^t)$  of a composition node (2) incorporates the order selectivity  $\omega$  and the selectivity  $D_{shared}$  for overlapping views. Both factors account for requirements that the input views of the composition do not enforce.

**The Order Selectivity  $\omega$ .** Every view in the effective view set  $S^+$  enforces the sequential order among its own variables, i.e., each view contributes a chain of variables ordered by pattern position. Between views, no order is enforced. The composition only accounts for the interleaving of already ordered chains. The union of the chains, with shared variables identified, forms a partial order over the variables of the pattern. Based on the uniformity assumption (Appendix A), every interleaving of the chains is equally likely. These interleavings correspond to the linear extensions of the partial order, and exactly one of them is the pattern's sequential order. With  $\text{ext}(S^+)$  denoting the number of linear extensions, the order selectivity is given by:

$$\omega = \frac{1}{\text{ext}(S^+)}. \quad (6)$$

For pairwise disjoint views, the number of linear extensions is given by the multinomial coefficient: an interleaving of chains with lengths  $k_1, \dots, k_r$  is determined by choosing the positions each chain occupies, such that  $\text{ext}(S^+) = (\sum_i k_i)! / \prod_i k_i!$ . For overlapping views, the chains are connected at the shared variables, which reduces the number of linear extensions. In this case, we count the linear extensions with a standard dynamic program over subsets of variables, which is feasible for the actually encountered pattern sizes.

**EXAMPLE 9.** Consider the effective view set  $S^+ = \{(A, O), (A, S), (B^+)\}$  of the running example, whose join graph is shown in Fig. 6a: the chains  $A < O$  and  $A < S$  share the variable  $A$ , while  $B$  is unconstrained. Among the  $4! = 24$  orderings of  $\{A, O, B, S\}$ , the 8 orderings placing  $A$  before  $O$  and  $S$  are the linear extensions, such that  $\omega = \frac{1}{8}$ . Exactly one of them,  $A < O < B < S$ , is the pattern's sequential order. Under the all-singleton strategy, every chain consists of a single variable and  $\omega = \frac{1}{24}$ , i.e., the composition enforces the complete sequential order.

**The Selectivity  $D_{shared}$ .** Overlapping views constrain more than the order: their entries can only be joined if every shared variable is assigned the same row by all views containing it. Consider a variable  $V$  contained in  $m_V$  views of  $S^+$ . Each view assigns to  $V$

one of the  $|T_t| \varphi_V$  rows satisfying  $C_V$ . Assuming each of these rows to be assigned with equal probability, an additional occurrence of  $V$  agrees with the first one with probability  $1 / (|T_t| \varphi_V)$ . Multiplying over all occurrences and all shared variables yields:

$$D_{shared} = \prod_{V: m_V \geq 2} (|T_t| \varphi_V)^{-(m_V-1)}. \quad (7)$$

Here, the product ranges over the shared variables, i.e., all variables contained in more than one view of  $\mathcal{S}^+$ ; every occurrence beyond the first contributes one factor. In the example above,  $A$  is shared by  $(A, O)$  and  $(A, S)$ , such that  $D_{shared} = (|T_t| \varphi_A)^{-1}$ . During execution, this requirement corresponds to the equi-join on the row identifier of  $A$  ( $\tau = \text{EQUI}$ , Definition 6). Note that the two factors do not double count:  $D_{shared}$  reflects the requirement that shared variables are assigned the same rows, while  $\omega$  reflects the reduced number of orderings resulting from the connected chains.

## C THE CONSTANT $\kappa$ AND THE CYCLIC CORRECTION $\beta(S)$

**The Constant  $\kappa$ .** The materialization strategy objective  $\Phi$  (3) combines a maintenance term and a composition term, which count rows in different phases of the execution. The maintenance term counts rows processed by the update joins across the  $K$  batches. The output of each update join is materialized, i.e., it extends the stored state of the view. The composition term counts rows processed by the composition joins during the  $|\mathcal{R}|$  executions of the query phase; these rows are not materialized. Since maintaining a row is substantially more expensive than processing it during composition, the two terms are not directly comparable. The constant  $\kappa$  therefore expresses the costs of maintaining one prefix row for one batch in units of composition work, analogous to the physical calibration of the cost model (§5.4). We calibrate  $\kappa$  once per engine: we fit per-row costs for maintenance and composition based on measured executions of plans with known cardinalities and set  $\kappa$  to their ratio; in our setup,  $\kappa \approx 150$ . Note that the absolute scale of  $\Phi$  does not influence the selection, as only the relative costs of maintenance and composition can affect the selected strategy.

**The Cyclic Correction  $\beta(S)$ .** For a view  $S$  whose internalized edges  $E(S)$  contain a cycle, the estimate  $n_{\text{pre}(S)}(K)$  underestimates the update costs. Every internalized edge contributes its selectivity  $\sigma_e(K)$  to the final cardinality of the view, such that the state of a view with many internalized conditions is small. However, a join prevents combinations from being formed only along the edges it joins on. Assembling the view requires  $|S| - 1$  joins, and an edge can serve as a join predicate only at the step in which its two variables first appear together. Consequently, at most a spanning tree of  $E(S)$  is applied while combinations are formed. The remaining  $c = |E(S)| - (|S| - 1)$  closing edges, denoted by  $\text{close}(S)$ , are evaluated as filters on the intermediate results: they discard combinations after the join has processed them. Each closing edge  $e$  therefore increases the intermediate results by a factor of  $1/\sigma_e(K)$  compared to the final cardinality. An optimal join order uses the most selective edges as join predicates and defers the least selective ones, which yields the correction:

$$\beta(S) = \prod_{e \in \text{close}(S)} \frac{1}{\sigma_e(K)}. \quad (8)$$

Here,  $\text{close}(S)$  contains the  $c$  least selective edges of  $E(S)$ ; for acyclic  $E(S)$ , it holds  $\beta(S) = 1$ .

**EXAMPLE 10.** Consider a view over three plain variables with all three pairwise dependent conditions, each with selectivity  $\sigma_e(K) = 0.02$ . Two conditions serve as join predicates and the third one closes the triangle, such that  $c = 1$  and  $\beta(S) = 50$ , i.e., the update join processes intermediate results that are fifty times larger than the final view. Without  $\beta(S)$ , the score of such views would be underestimated, since every internalized edge reduces the final cardinality by its selectivity, and the search would consolidate views although their intermediate results grow.